KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 1
붕어빵 틀과 붕어빵

틀 하나로 붕어빵을
여러 개 찍어 내기

```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
```

오늘의 순서

1 왜 필요한가

딕셔너리와 함수의 불편함, 틀과 붕어빵 그림, 이미 써 온 틀

4–6쪽

2 가장 작은 틀

class와 pass, 점으로 값 붙이기, 손으로 붙일 때의 문제

7–9쪽

3 self와 __init__

함수에서 __init__으로, self 따라가기, 왼쪽과 오른쪽

10–20쪽

4 메서드

intro 만들기, Bread.intro(a), 값 바꾸기와 return, 비교

21–24쪽

5 오류와 함정

재료 수, self 빠짐, __init__ 이름, print 결과

25–28쪽

6 실습

붕어빵 주문, 카페 음료 이름표, 도장 카드

29–31쪽

3쪽은 노트북 준비, 32–33쪽은 막힐 때와 한 장 요약이에요.

들어가기

23에서 이어서 해요

- ✓ 변수, 타입, 함수 type() 12
- ✓ 리스트, f-string, for 13
- ✓ 딕셔너리 15
- ✓ def, 매개변수, return 19
- ✓ 기본값, 지역 변수 20

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

1. 왜 필요한가

붕어빵을 딕셔너리로 담으면 값과 할 일이 따로 놀아요

[1]
```python
b1 = {"flavor": "팥", "price": 1000}
b2 = {"flavor": "슈크림", "price": 1200}

def intro(bread):
    print(bread["flavor"], "붕어빵", bread["price"], "원")

intro(b1)
intro(b2)
```

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
```

[2]
```python
b3 = {"flavr": "피자", "price": 1500}
intro(b3)
```

```
KeyError: 'flavor'
```

값은 딕셔너리, 할 일은 함수

붕어빵 정보와 붕어빵 소개 함수가 서로 다른 곳에 있어요.

키를 틀려도 모르고 넘어가요

b3를 만들 때는 아무 말이 없다가, 소개할 때 KeyError가 나요.

값과 할 일을 한 틀에 묶어 두는 방법이 클래스예요.

1. 왜 필요한가

틀 하나로 붕어빵을 여러 개 찍어요

틀 class Bread

맛 flavor = ?
가격 price = ?

할 일 intro()

팥, 1000

슈크림, 1200

a — Bread 붕어빵

flavor	"팥"
price	1000

b — Bread 붕어빵

flavor	"슈크림"
price	1200

틀은 하나

틀에는 맛과 가격을 넣을 자리만 있어요. 붕어빵마다 다른 값이 들어가요.

틀 = 클래스 class로 만들어요

붕어빵 = 객체 틀로 찍어 낸 것 하나

속성 붕어빵에 든 값 (맛, 가격)

메서드 붕어빵이 하는 일 (소개하기)

1. 왜 필요한가

사실 int, str, list도 틀이에요

[3]
```
print(type(3))
print(type("팥"))
print(type([1, 2]))
```
Python

```
<class 'int'>
<class 'str'>
<class 'list'>
```

class라고 나와요

12에서 본 type()의 결과예요. 3은 int 틀, "팥"은 str 틀, [1, 2]는 list 틀로 찍은 객체예요.

[4]
```
xs = [1, 2]
xs.append(3)
print(xs)
print("Pat".lower())
```
Python

```
[1, 2, 3]
pat
```

점 뒤의 함수가 메서드예요

13의 append, 22의 lower가 메서드였어요. 리스트 틀, 문자열 틀에 미리 들어 있는 할 일이에요.

이미 써 온 틀처럼, 붕어빵 틀을 직접 만들어 봐요.

2. 가장 작은 틀

빈 틀을 만들고 붕어빵 두 개를 찍어요

[1]
```python
class Bread:
    pass

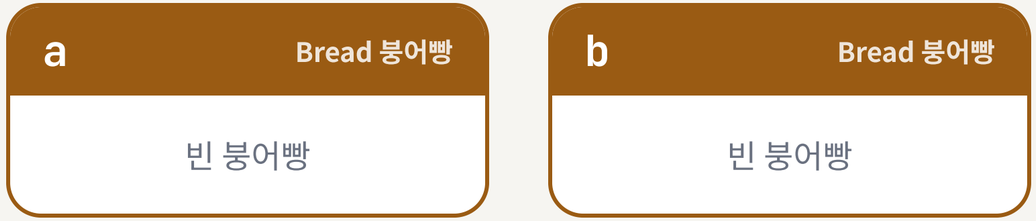
a = Bread()
b = Bread()
print(type(a))
```

```
<class '__main__.Bread'>
```

`class Bread:` 틀 이름이에요. 첫 글자는 대문자로 적어요

`pass` 아직 아무것도 안 넣었다는 표시예요

`Bread()` 틀로 붕어빵 하나 찍기. 괄호를 꼭 붙여요



__main__.Bread

지금 노트북(__main__)에서 만든 Bread 틀로 찍었다는 뜻이에요.

2. 가장 작은 틀

점(.)으로 붕어빵에 값을 붙여요

[2]
```python
a.flavor = "팥"
a.price = 1000
b.flavor = "슈크림"
b.price = 1200
print(a.flavor, a.price)
print(b.flavor, b.price)
```

```
팥 1000
슈크림 1200
```



	딕셔너리 (15)	붕어빵 (객체)
값 넣기	`b1["flavor"] = "팥"`	`a.flavor = "팥"`
값 꺼내기	`b1["flavor"]`	`a.flavor`

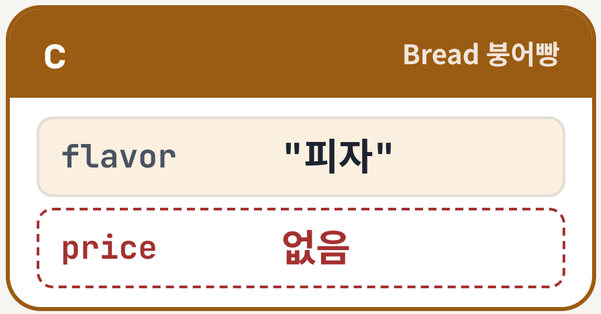
a.flavor는 "a의 flavor"로 읽어요.

2. 가장 작은 틀

손으로 붙이면 길고, 하나 빠뜨리기 쉬워요

[3]
```python
c = Bread()
c.flavor = "피자"
print(c.price)
```

```
AttributeError: 'Bread' object has no attribute 'price'
```



price를 빠뜨렸어요

c에는 price 칸이 없어서 AttributeError가 나요. 붕어빵에 그런 속성이 없다는 뜻이에요.

붕어빵이 10개라면

찍기 10줄, 맛 10줄, 가격 10줄을 적어야 해요. 한 줄만 빠져도 그 붕어빵은 칸이 비어요.

찍는 순간 재료를 한 번에 넣게 하면 빠뜨릴 일이 없어요.

3. self와 __init__

먼저 19에서 배운 함수로 한 번에 붙여 봐요

[1]
```python
class Bread:
    pass

def setup(bread, flavor, price):
    bread.flavor = flavor
    bread.price = price

a = Bread()
setup(a, "팥", 1000)
print(a.flavor, a.price)
```
붙이는 함수

Python

```
팥 1000
```

```
setup(a, "팥", 1000)
```

bread ← a 어느 붕어빵에 붙일지

flavor ← "팥" 넣을 맛

price ← 1000 넣을 가격

bread.flavor = flavor

a에 맛 칸을 붙이는 것과 같아요.

3. self와 __init__

그 함수를 틀 안에 넣으면 __init__이에요

밖에 둔 함수

```
def setup(bread, flavor, price):
    bread.flavor = flavor
    bread.price = price

a = Bread()
setup(a, "팥", 1000)
```

```
팥 1000
```

틀 안의 __init__

```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
```

```
팥 1000
```

들여쓰기 class 아래로 넣어요

setup → __init__ 찍을 때 저절로 불리는 약속된 이름

bread → self 이름만 바뀌고 하는 일은 같아요

Bread("팥", 1000) a를 넘기지 않아요. 파이썬이 넣어 줘요

3. self와 __init__

괄호 안의 재료가 __init__의 칸에 들어가요

```
a = Bread("팥", 1000)            # 찍는 줄
def __init__(self, flavor, price):  # 틀 안의 첫 줄
```

파이썬이 넣어 줘요

괄호 안의 재료가 순서대로 (1의 매개변수)

새 붕어빵	"팥"	1000
↓	↓	↓
self	flavor	price

self는 적지 않아요

칸은 세 개, 재료는 두 개예요. 첫 칸은 파이썬 몫이에요.

__init__ 이름

앞뒤로 밑줄 두 개씩. init은 처음 준비한다는 뜻의 initialize예요.

3. self와 __init__

__init__은 찍는 순간 저절로 불려요

[3]

```python
class Bread:
    def __init__(self, flavor, price):
        print("반죽 붓는 중:", flavor)          # 확인용
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)                          # 찍을 때마다
b = Bread("슈크림", 1200)
print(a.flavor, b.flavor)
```

Python

```
반죽 붓는 중: 팥
반죽 붓는 중: 슈크림
팥 슈크림
```

직접 부르지 않아요

a.__init__()이라고 치지 않았는데, 찍는 줄마다 한 번씩 출력됐어요.

언제 쓰나요

붕어빵마다 꼭 있어야 하는 값을 찍는 순간 넣어 둘 때 써요.

3. self와 __init__

self는 지금 찍고 있는 그 붕어빵이에요

```
a = Bread("팥", 1000)
```

```
b = Bread("슈크림", 1200)
```

```
def __init__(self, flavor, price):
    self.flavor = flavor
    self.price = price
```



a를 찍는 동안 self는 a를 가리켜요

self.flavor = flavor는 지금은 a.flavor = "팥"과 같아요.

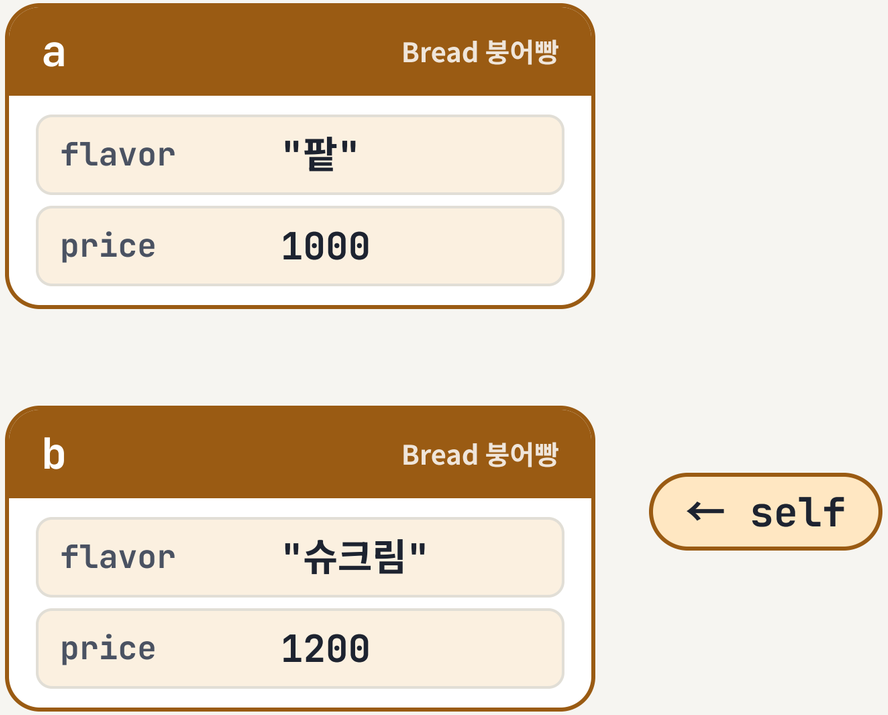
3. self와 __init__

self는 지금 찍고 있는 그 붕어빵이에요

```
a = Bread("팥", 1000)
```

```
b = Bread("슈크림", 1200)
```

```
def __init__(self, flavor, price):
    self.flavor = flavor
    self.price = price
```



첫째 줄을 실행할 때 self는 a, 둘째 줄을 실행할 때 self는 b예요

그래서 __init__ 코드 한 벌로 붕어빵마다 다른 값을 붙일 수 있어요.

3. self와 __init__

a = Bread("팥", 1000) 한 줄에서 일어나는 일

① **Bread("팥", 1000)**
빈 붕어빵이 하나 생겨요

② **__init__(빈 붕어빵, ...)**
빈 붕어빵이 self로 들어가요

③ **self.flavor = "팥"**
맛 칸을 붙여요

④ **self.price = 1000**
가격 칸을 붙여요

⑤ **a = 다 된 붕어빵**
끝나면 a에 담아요

a — Bread 붕어빵

flavor	"팥"
price	1000

self는 누구?

②부터 ④까지 self는 방금 생긴 그 붕어빵이에요. ⑤에서 그 붕어빵에 a라는 이름이 붙어요.

3. self와 __init__

눌러 보기: 붕어빵 기계로 한 단계씩 찍어 봐요

맛 팥 슈크림 피자 고구마

가격 1000 1200 1500

틀 Bread

__init__(self, flavor, price)

찍으면 여기서 붕어빵이 만들어져요

```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
```

찍은 붕어빵

아직 없어요. 찍으면 여기에 쌓여요.

한 단계씩 ▶ 한 번에 찍기 처음부터

맛과 가격을 고르고 **한 단계씩**이나 **한 번에 찍기**를 눌러요. 브라우저로 슬라이드를 열면 눌러져요.

3. self와 __init__

self.flavor = flavor는 왼쪽과 오른쪽이 달라요

```
self.flavor = flavor
```

self.flavor

붕어빵에 붙는 칸

붕어빵이 있는 동안 계속 남아요. 나중에 a.flavor로 꺼내요.

flavor

찍을 때 들어온 재료

__init__의 매개변수예요. 20의 지역 변수라서 __init__이 끝나면 사라져요.

이름이 같아서 헷갈려요

self. 이 붙은 쪽은 붕어빵 칸, 안 붙은 쪽은 잠깐 들고 온 재료예요. 보통은 같은 이름을 써요.

3. self와 __init__

self.를 빼면 재료가 붕어빵에 안 붙어요

[4]
```python
class Bread:
    def __init__(self, flavor, price):
        flavor = flavor                    # self. 없음
        price = price

a = Bread("팥", 1000)
print(a.flavor)
```

Python

```
AttributeError: 'Bread' object has no attribute 'flavor'
```

a — Bread 붕어빵

붙은 칸이 없어요

지역 변수에 다시 넣었을 뿐이에요

flavor = flavor는 재료를 자기 자신에 다시 넣었어요.
__init__이 끝나면 같이 사라져요.

붕어빵에 남길 값은 꼭 self.이름 = 값으로 적어요.

3. self와 __init__

재료로 받지 않는 칸도 만들 수 있어요

[5]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        self.baked = False                    # 처음 값

a = Bread("팥", 1000)
print(a.flavor, a.price, a.baked)
```

```
팥 1000 False
```

a — Bread 붕어빵

flavor	"팥"
price	1000
baked	False

언제 쓰나요

처음에는 모두 같은 값으로 시작하는 칸이에요. 다 구웠는지, 도장 0개, 판매 0개 같은 값이에요.

4. 메서드

틀 안의 함수가 메서드예요

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro(self):                                    메서드
        print(self.flavor, "붕어빵", self.price, "원")

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a.intro()
b.intro()
```

Python

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
```

intro(self) 틀 안에 들여 쓴 함수. 첫 칸은 self

self.flavor 그 붕어빵의 맛을 꺼내요

a.intro() a에게 소개하라고 시켜요. 괄호는 비워요

1파트의 intro와 비교

intro(b1)처럼 넘기지 않고, 붕어빵에게 a.intro()로 시켜요.

4. 메서드

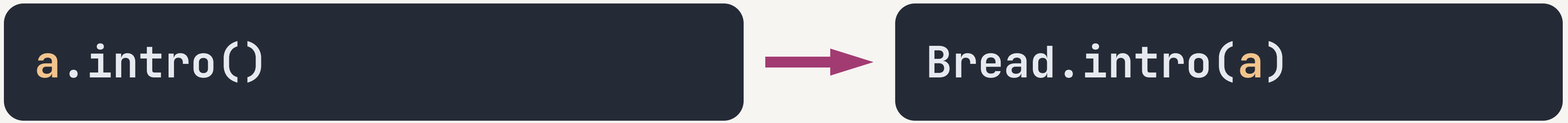
a.intro()는 Bread.intro(a)와 같아요

[2]
```python
Bread.intro(a)
Bread.intro(b)
```

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
```

결과가 똑같아요

앞 셀의 a.intro(), b.intro()와 같은 두 줄이 나왔어요.



괄호가 비어 있는데 self 칸이 있는 이유

점 앞의 a가 self 칸으로 들어가요. 파이썬이 a.intro()를 Bread 틀의 intro에 a를 넣어 부르는 걸로 바꿔요. __init__에서 방금 찍은 붕어빵이 self로 들어간 것과 같아요.

4. 메서드

메서드로 값을 바꾸고, return으로 돌려받아요

[3]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def discount(self, won):                    # 값 바꾸기
        self.price = self.price - won

    def total(self, qty):                       # 돌려주기
        return self.price * qty

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a.discount(200)
print(a.price, b.price)
print(a.total(3))
```

Python

```
800 1200
2400
```

a — Bread 붕어빵

flavor	"팥"
price	1000 → 800

b — Bread 붕어빵

flavor	"슈크림"
price	1200

a만 바뀌어요

a.discount(200)의 self는 a예요. b의 가격은 그대로예요.

4. 메서드

딕셔너리와 함수 vs 클래스

딕셔너리와 함수 (1파트)

```
b1 = {"flavor": "팥", "price": 1000}
intro(b1)
b1["flavor"]
```

클래스

```
a = Bread("팥", 1000)
a.intro()
a.flavor
```

	딕셔너리와 함수	클래스
할 일은 어디에	함수가 밖에 따로 있어요	틀 안에 같이 있어요
값 꺼내기	`b1["flavor"]`	`a.flavor`
재료를 빠뜨리면	쓸 때 KeyError	찍을 때 바로 TypeError
어울리는 곳	잠깐 담아 두는 값	같은 모양을 여러 개 찍을 때

5. 오류와 함정

재료 수가 모자라면 missing argument

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro():
        print("붕어빵")

a = Bread("팥")
```
재료 하나 Python

```
TypeError: Bread.__init__() missing 1 required positional argument: 'price'
```

읽는 법

__init__에 꼭 필요한 칸 price가 1개 모자라요.

고치기

Bread("팥", 1000)처럼 self 뒤 칸 수만큼 재료를 넣어요.

5. 오류와 함정

메서드에 self를 빼면 takes 0 positional arguments

앞 셀의 틀 일부

```
    def intro():
        print("붕어빵")
```

[2]

```
a = Bread("팥", 1000)
a.intro()
```

Python

```
TypeError: Bread.intro() takes 0 positional arguments but 1 was given
```

읽는 법

intro는 받을 칸이 0개인데 1개가 들어왔어요. 그 1개는 점 앞의 a예요.

고치기

메서드 첫 칸에 self를 꼭 적어요. def intro(self):

a.intro()는 Bread.intro(a)라서 a가 하나 들어가요.

5. 오류와 함정

밑줄을 하나만 적으면 takes no arguments

[3]
```python
class Bread:
    def _init_(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
```

init

Python

```
TypeError: Bread() takes no arguments
```

읽는 법

Bread 틀은 재료를 받지 않는다는 뜻이에요. 약속된 이름 __init__을 찾지 못했어요.

고치기

앞뒤로 밑줄 두 개씩 __init__. __int__처럼 글자가 빠져도 같은 오류가 나요.

5. 오류와 함정

붕어빵을 print하면 이상한 글자가 나와요

[4]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
print(a)
```

```
<__main__.Bread object at 0x7f440c2dbb60>
```

__main__.Bread 지금 노트북에서 만든 Bread 틀의

object 객체(붕어빵)예요

at 0x7f… 컴퓨터 안의 자리. 실행할 때마다 달라요

오류는 아니에요

파이썬이 이 붕어빵을 글자로 어떻게 보여 줄지 몰라서 기본 모양으로 찍었어요.

25에서 바꿔요

지금은 a.flavor처럼 칸을 꺼내 찍어요. 다음 챕터의 __str__로 보기 좋게 바꿔요.

6. 실습

붕어빵 가게: 주문 받은 붕어빵을 소개하고 합계 내기

[1]

```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

order = [Bread("팥", 1000), Bread("슈크림", 1200), Bread("팥", 1000)]
for b in order:                                                    하나씩
    b.intro()
total = sum([b.price for b in order])
print("합계", total, "원")
```

Python

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
팥 붕어빵 1000 원
합계 3200 원
```

객체도 값이에요

리스트에 담고 for로 하나씩 꺼내요.

b.price만 모아요

23의 컴프리헨션으로 가격을 모아 sum에 넘겨요.

6. 실습

카페 음료: 기본값 사이즈와 메뉴 이름표 만들기

[2]

```python
class Drink:
    def __init__(self, name, price, size="M"):          # 20 기본값
        self.name = name
        self.price = price
        self.size = size

    def label(self):
        return f"{self.name}({self.size}) {self.price:,}원"          # return

d1 = Drink("latte", 5000)
d2 = Drink("tea", 4000, "L")
print(d1.label())
print(d2.label())
```

Python

```
latte(M) 5,000원
tea(L) 4,000원
```

d1 Drink

name	"latte"
price	5000
size	"M"

d2 Drink

name	"tea"
price	4000
size	"L"

label은 글자를 돌려줘요

print는 밖에서 해요. 영수증에 다시 쓸 수 있어요.

6. 실습

도장 카드: 처음 0개에서 손님마다 따로 쌓기

[3]

```python
class Card:
    def __init__(self, name):
        self.name = name
        self.stamps = 0          # 처음 값

    def add(self):
        self.stamps = self.stamps + 1   # 하나 더하기

kim = Card("kim")
lee = Card("lee")
kim.add()
kim.add()
kim.add()
lee.add()
print(kim.name, kim.stamps)
print(lee.name, lee.stamps)
```

Python

```
kim 3
lee 1
```

kim — Card

name	"kim"
stamps	0 → 3

lee — Card

name	"lee"
stamps	0 → 1

self 덕분에 따로 쌓여요

kim.add()는 kim의 도장만, lee.add()는 lee의 도장만 늘려요.

정리

막히면 오류의 마지막 줄을 봐요

1	`TypeError: ... missing 1 required positional argument` 찍을 때 재료가 모자라요	self 뒤 칸 수만큼
2	`TypeError: ... takes 0 positional arguments but 1 was given` 메서드에 self가 없어요	`def intro(self):`
3	`TypeError: Bread() takes no arguments` __init__ 이름이 틀렸어요	밑줄 두 개씩
4	`AttributeError: 'Bread' object has no attribute ...` 칸 이름 오타이거나 self.를 뺐어요	self.이름 = 값
5	`NameError: name 'flavor' is not defined` 메서드 안에서 self.flavor 대신 flavor를 썼어요	`self.flavor`
6	`<__main__.Bread object at 0x...>` 붕어빵을 통째로 print했어요	a.flavor로, 25의 __str__
7	`TypeError: ... missing 1 required positional argument: 'self'` a = Bread처럼 괄호 없이 틀을 담았어요	Bread(재료)

헷갈리면 self 자리에 붕어빵 이름(a)을 넣어 소리 내어 읽어 봐요.

한 장 요약

`class Bread:`

틀 만들기

Bread("팥", 1000)

붕어빵 찍기

`def __init__(self, ...)`

찍을 때 재료 넣기

`self.flavor = flavor`

붕어빵에 칸 붙이기

`self`

지금 그 붕어빵

`def intro(self):`

메서드 만들기

`a.intro()`

Bread.intro(a)와 같아요

`a.flavor`

속성 꺼내기

self는 "방금 찍은 붕어빵", "메서드를 부른 붕어빵"이에요

다음 단계: 25에서 __str__, __repr__로 붕어빵을 보기 좋게 찍어요